Documentação Técnica do Código GNN — Previsão de Anomalias Climáticas

> Análise teórica completa do pipeline de Graph Neural Network (GNN) do projeto. Código de referência: `grafo-0.8.0-pyg.ipynb` (versão de produção) + `dashboard.py` (consumo dos artefatos). Esta documentação explica **o que cada bloco de código faz**, a **teoria por trás** e, sempre que aparece, **como funciona cada função de biblioteca que não é nativa do Python** (NetworkX, PyTorch, PyTorch Geometric, pandas, NumPy, Streamlit/Plotly).

Índice

1. Visão geral do pipeline
2. Evolução das versões (0.09 → 0.8.0)
3. Bloco 1 — Imports e seleção de dispositivo
4. Bloco 2 — Carregamento do grafo
5. Bloco 3 — Geografia: coordenadas, haversine e pesos de aresta
6. Bloco 4 — Leitura robusta dos CSVs
7. Bloco 5 — Mapeamento estação→nó (fuzzy matching)
8. Bloco 6 — Montagem do tensor `[T, N, F]`
9. Bloco 7 — Normalização das arestas (Mudança 1)
10. Bloco 8 — Grafo PyTorch Geometric
11. Bloco 9 — Feature Propagation (Mudança 2)
12. Bloco 10 — Janelamento temporal e máscara de loss
13. Bloco 11 — A camada `WeightedSAGEConv` e o modelo GraphSAGE
14. Bloco 12 — Treino
15. Bloco 13 — Avaliação e gráficos
16. Bloco 14 — Identificação de anomalias (Mudança 3)
17. Bloco 15 — Métricas e baseline
18. Bloco 16 — Exportação de artefatos
19. `dashboard.py` — visualização
20. Glossário de funções de biblioteca

---

1. Visão geral do pipeline

O sistema resolve um problema de **previsão espaço-temporal em grafo**: dada a leitura de ~35 estações meteorológicas de São Paulo (nós) ligadas por proximidade geográfica (arestas), prever a variável de cada estação no próximo instante (~10 min à frente) e detectar anomalias.

O grande desafio é que **3 estações não têm dados próprios** (os "nós cegos": IAG-USP, INMET automático e INMET convencional, índices 33, 34, 35). Eles precisam ser estimados a partir dos vizinhos.

Fluxo completo:

```
001.graphml ──► NetworkX (estrutura do grafo)
   │
   │  + coordenadas DMS/decimais → haversine → peso = 1/distância
   ▼
CSVs 2023/2024 ──► leitura robusta ──► mapeamento estação→nó
   │
   ▼
Tensor X [T, N, F]   (T = tempo, N = nós, F = 4 features: PLU, Temp, Umid, Vel)
   │
   ├─► Normalização de arestas D⁻¹A  (Mudança 1)
   ├─► Feature Propagation nos nós cegos  (Mudança 2, Rossi et al. 2022)
   ▼
GraphSAGE (2 camadas, WeightedSAGEConv)  ── treino em 2023, teste em 2024
   │                                          (2 modelos: PLU e Temp)
   ▼
Previsões x̂(t+h)
   │
   ├─► Regra 1: limiar P95 por mês/estação + janela anti-ruído  (Mudança 3)
   ├─► Regra 2: anomaly score de Mahalanobis
   ▼
Artefatos (.pt, .pkl) ──► dashboard.py (Streamlit + Plotly)
```

A nomenclatura "Mudança 1/2/3" vem das células markdown do próprio notebook, que descrevem as três grandes novidades da versão 0.8.0.

---

2. Evolução das versões

O repositório contém vários notebooks que documentam a história do projeto. **Apenas o 0.8.0 é a versão de produção**; os demais ficam como registro:

Notebook	Framework	Marco
`grafo-0.09` → `0.5-dgl`	**DGL** (Deep Graph Library)	Primeiras GNNs, ainda na biblioteca DGL
`grafo-0.6-pyg` em diante	**PyTorch Geometric (PyG)**	Migração para PyG
`0.7.x-pyg`	PyG	Refinamentos: nós cegos, pesos haversine, separação treino/teste
`0.8.0-pyg`	PyG	**Atual**: normalização D⁻¹A, Feature Propagation, detecção de anomalias, exportação para dashboard

A migração **DGL → PyG** é relevante: ambas são bibliotecas de GNN, mas o projeto padronizou no PyG (mais usado academicamente e com a API `MessagePassing` que o código explora). Toda a análise abaixo é sobre o PyG.

3. Bloco 1 — Imports e seleção de dispositivo

```
def selecionar_dispositivo():
    if torch.cuda.is_available():
        return torch.device("cuda"), "cuda"
    if getattr(torch.backends, "mps", None) is not None and
torch.backends.mps.is_available():
        return torch.device("mps"), "mps"
    return torch.device("cpu"), "cpu"
```

O que faz: escolhe o melhor backend de hardware disponível, em ordem de preferência: **CUDA** (GPU NVIDIA) → **MPS** (GPU Apple Silicon, Metal) → **CPU**.

Funções de biblioteca não-nativas: - `torch.cuda.is_available()` — retorna `True` se há GPU NVIDIA com driver CUDA acessível ao PyTorch. - `torch.backends.mps.is_available()` — verifica o backend **Metal Performance Shaders** da Apple. O `getattr(..., None)` é uma proteção: versões antigas do PyTorch não têm o atributo `mps`, e acessá-lo direto quebraria. O código checa a existência antes de usar. - `torch.device("cuda" | "mps" | "cpu")` — cria um objeto que representa "onde" os tensores vivem. Tensores e modelos são movidos para lá com `.to(DEVICE)`.

Em seguida cria a pasta `artifacts/` com `os.makedirs(ARTIFACTS_DIR, exist_ok=True)` (`exist_ok=True` evita erro se a pasta já existir).

---

4. Bloco 2 — Carregamento do grafo

```
G = nx.read_graphml("001.graphml")
```

`nx.read_graphml` (NetworkX): lê um arquivo no formato **GraphML** (um XML padronizado para grafos) e devolve um objeto grafo do NetworkX. Cada nó e cada aresta podem ter atributos (aqui: `label`, coordenadas `x` / `y`, etc.). O grafo pode ser um `MultiGraph` / `MultiDiGraph` — por isso o código depois itera arestas com `keys=True`.

`G.number_of_nodes()`, `G.number_of_edges()`, `G.is_directed()`, `G.nodes()` são acessores básicos do NetworkX que retornam, respectivamente, contagem de nós, de arestas, se é direcionado e a lista de nós.

---

5. Bloco 3 — Geografia: coordenadas, haversine e pesos de aresta

5.1 Conversão de coordenadas

```
def dms_to_decimal(degrees, minutes, seconds, direction):
    decimal = float(degrees) + float(minutes) / 60 + float(seconds) / 3600
    if str(direction).upper() in ["S", "W", "O"]:
```

```
        decimal *= -1
    return decimal
```

Converte coordenadas no formato **DMS** (graus, minutos, segundos) para **graus decimais**. Sul (S) e Oeste (W/O) viram negativos — São Paulo fica no hemisfério Sul e a Oeste de Greenwich, então todas as lat/lon são negativas. As coordenadas estão hard-coded em `_COORDS_DMS` (formato DMS) e `_COORDS_DEC` (já decimal, nós 0 e 33).

5.2 Distância haversine

```
def haversine(lat1, lon1, lat2, lon2):
    R = 6371
    lat1, lon1, lat2, lon2 = map(math.radians, [...])
    dlat = lat2 - lat1; dlon = lon2 - lon1
    a = math.sin(dlat/2)**2 + math.cos(lat1)*math.cos(lat2)*math.sin(dlon/2)**2
    c = 2 * math.asin(math.sqrt(a))
    return R * c
```

Teoria: a **fórmula de haversine** calcula a distância de "grande círculo" (sobre a superfície da esfera terrestre) entre dois pontos de latitude/longitude. `R = 6371` km é o raio médio da Terra. É mais correta que a distância euclidiana para coordenadas geográficas. `math.radians` converte graus → radianos (exigência das funções trigonométricas).

5.3 Peso das arestas = inverso da distância

```
DIST_MIN_KM = 0.1
dist_km = max(dist_km, DIST_MIN_KM)
inv_dist = 1.0 / dist_km
G.edges[u, v, k]['weight'] = inv_dist
```

Teoria (homofilia espacial): estações próximas tendem a ter clima parecido, então o peso da aresta é o **inverso da distância** — vizinhos próximos influenciam mais a mensagem propagada na GNN. O `DIST_MIN_KM = 0.1` é um **piso de segurança**: o par CGE-Butantã ↔ IAG-USP fica a ~0.1 km; sem o piso, `1/distância` explodiria e essa aresta dominaria toda a agregação. (Esse problema motivou a Mudança 1, ver Bloco 7.)

`G.edges(keys=True)` itera arestas incluindo a chave `k` do multigrafo. `G.edges[u, v, k]['attr'] = ...` grava um atributo na aresta.

5.4 Visualização

```
pos = {n: (float(G.nodes[n]['x']), float(G.nodes[n]['y'])) for n in G.nodes}
nx.draw(G, pos, with_labels=True, node_size=400, font_size=8)
```

`nx.draw` desenha o grafo com matplotlib, posicionando cada nó nas coordenadas `x` / `y` armazenadas no GraphML.

---

6. Bloco 4 — Leitura robusta dos CSVs

Os CSVs do CGE vêm "sujos" (separadores variados, cabeçalho quebrado, vírgula decimal, datas em formatos mistos). Três funções tratam isso:

```
def _detectar_sep(caminho_csv):
    linha = f.readline()
    return ";" if linha.count(";") > linha.count(",") else ","
```

Detecta o separador olhando a primeira linha (conta `;` vs `,`).

```
def _corrigir_cabecalho_csv_cge(caminho_csv):
    cabecalho_corrigido = cabecalho_original.replace("DATA,,", "DATA,", 1)
```

Corrige um bug específico do CGE: o cabeçalho vem com uma vírgula dupla (`DATA,,`), que geraria uma coluna fantasma `Unnamed: 2`. A correção é feita **no texto bruto, antes do parse** — o código inclusive levanta erro se `Unnamed: 2` ainda aparecer depois, como guarda de sanidade.

`carregar_estacoes_sem_agregar` — funções pandas relevantes

```
df = pd.read_csv(io.StringIO(conteudo_csv), sep=sep, engine="python",
                 encoding="utf-8", on_bad_lines="skip")
```

- **`io.StringIO`** (invocado via `__import__("io").StringIO`): transforma a *string* já corrigida em um "arquivo em memória", permitindo que `pd.read_csv` leia o texto corrigido em vez do arquivo no disco.
- `engine="python"` — parser mais flexível (necessário para separadores irregulares).
- `on_bad_lines="skip"` — pula linhas malformadas em vez de quebrar.

```
df["DATA"] = pd.to_datetime(df["DATA"], errors="coerce", format="mixed", dayfirst=True)
```

`pd.to_datetime` converte texto em timestamps. `format="mixed"` aceita múltiplos formatos de data na mesma coluna; `dayfirst=True` interpreta `dd/mm/aaaa` (padrão brasileiro); `errors="coerce"` transforma datas inválidas em `NaT` (Not-a-Time), que depois são removidas com `dropna`.

```
df = df.dropna(subset=["DATA"]).set_index("DATA").sort_index()
df = df[~df.index.duplicated(keep="first")]
```

- `set_index("DATA")` torna a data o índice (eixo temporal).
- `sort_index()` ordena cronologicamente.
- `index.duplicated(keep="first")` marca timestamps repetidos; o `~` inverte para **manter** só a primeira ocorrência.

```
df[col] = df[col].astype(str).str.replace(",", ".", regex=False)
df[col] = pd.to_numeric(df[col], errors="ignore")
```

Troca vírgula decimal por ponto (`1,5` → `1.5`) e converte para número. O acessador `.str` aplica operações de string vetorizadas a toda a coluna.

```
df_final = pd.concat(estacoes_processadas).sort_index()
```

`pd.concat` empilha os DataFrames de todas as estações em um só (mantendo a coluna `station_id` que identifica a origem de cada linha).

---

7. Bloco 5 — Mapeamento estação→nó (fuzzy matching)

Os nomes dos arquivos CSV (ex.: `dados_cge___P503_Sé__CGE_`) precisam ser ligados aos nós do grafo (que têm `label` próprio). Como os nomes não batem exatamente, há um **casamento aproximado por tokens**.

```
def normalizar_nome(s):
    s = unicodedata.normalize("NFKD", s).encode("ASCII", "ignore").decode("ASCII")
    s = re.sub(r"[^a-z0-9]+", "_", s)
```

`unicodedata.normalize("NFKD", s)` decompõe caracteres acentuados em letra base + acento (ex.: `é` → `e` + `´`); o `.encode("ASCII", "ignore")` descarta os acentos. Resultado: texto sem acento, minúsculo, com separadores normalizados. `re.sub` (regex) substitui tudo que não é letra/dígito por `_`.

Estratégia de matching em duas etapas:

1. **`MAPA_FIXO_CSV`** — um dicionário de exceções resolvidas à mão (ex.: `"p503": "19"`). Tem prioridade: se o código da estação aparece no nome, retorna direto.
2. **Score por tokens** (`_palavras_relevantes` + `_score_tokens`): - extrai palavras relevantes do nome, removendo *stopwords* (`do`, `da`, `cge`, siglas de bairro como `bt`, `pi`, etc.) e códigos `p\d+`. - `_score_tokens` calcula `nº de tokens do label que casam / total de tokens do label`. O casamento é por substring nos dois sentidos (`pl in pc or pc in pl`). - aceita o melhor nó apenas se `score ≥ 0.5` (pelo menos metade dos tokens bateu).

```
NOS_CEGOS_DEFAULT = {"33", "34", "35"}
```

Define explicitamente os nós cegos. `imprimir_relacao_no_csv` produz um relatório legível de qual CSV foi para qual nó (e quais nós ficaram sem dados).

---

8. Bloco 6 — Montagem do tensor `[T, N, F]`

`preparar_dataset_final_v6` é a função central de pré-processamento. Transforma o DataFrame longo (uma linha por leitura) em um tensor 3D `[tempo, nós, features]`.

Seleção de colunas

`_selecionar_coluna_temperatura` escolhe a melhor coluna de temperatura disponível (preferindo `Temp(oC)`; cai para sensação térmica como último recurso) **pela contagem de valores não-nulos** — robustez contra esquemas diferentes entre anos. `_primeira_coluna_existente` faz o mesmo para PLU, vento e umidade, testando vários nomes alternativos.

Pivotagem (long → wide)

```
df_pivot = df_fix.reset_index().pivot_table(
    index="DATA", columns="node_id", values=features, aggfunc="mean"
).sort_index()
```

`pivot_table` é o coração da transformação. Reorganiza os dados para que: - cada **linha** seja um instante (`DATA`), - cada **coluna** seja o par `(feature, node_id)` — produzindo um índice de colunas **hierárquico** (`MultiIndex`), - `aggfunc="mean"` resolve colisões (se houver duas leituras no mesmo instante para o mesmo nó, tira a média).

```
df_pivot = df_pivot.reindex(
    columns=pd.MultiIndex.from_product([features, nodes_lista]),
    fill_value=np.nan)
```

`reindex` + **`MultiIndex.from_product`** força o DataFrame a ter **todas** as combinações `feature × nó` (mesmo as ausentes, preenchidas com `NaN`). Isso garante que o tensor final tenha shape consistente, com todos os nós do grafo presentes.

Imputação de faltantes (cascata)

```
df_pivot = df_pivot.interpolate(method="time", limit_direction="both")
df_pivot = inicializar_nos_cegos_zero(df_pivot, nos_cegos, features)
df_pivot = df_pivot.ffill().bfill()
```

- **`interpolate(method="time")`** preenche buracos respeitando o espaçamento temporal real do índice (interpolação proporcional ao tempo entre pontos válidos). `limit_direction="both"` interpola em ambos os sentidos.
- **`inicializar_nos_cegos_zero`** zera os nós cegos. Crucial: na 0.8.0 os nós cegos **não** são mais imputados por média aqui; eles entram como `0.0` e serão preenchidos depois pelo **Feature Propagation** (Bloco 9). O zero é a condição inicial do algoritmo de difusão.
- **`ffill()`** / **`bfill()`** (forward/backward fill) propagam o último/próximo valor válido para tampar buracos nas pontas.
- Por fim, PLU faltante vira `0.0` (sem chuva) e as demais variáveis recebem a média global.

Reshape para tensor

```
data_3d = df_pivot.values.reshape(len(df_pivot), len(features),
len(nodes_lista)).transpose(0, 2, 1)
X = torch.tensor(data_3d, dtype=torch.float32).to(DEVICE)
```

O `.values` extrai a matriz NumPy. Como as colunas estão ordenadas como `(feature, nó)`, o `reshape` produz `[T, F, N]` e o **`transpose(0, 2, 1)`** troca os eixos 1 e 2 para chegar no layout desejado **`[T, N, F]`** (tempo, nós, features). `torch.tensor(...).to(DEVICE)` cria o tensor PyTorch e o move para a GPU/CPU.

Resultado: `X_2023` (treino) e `X_2024` (teste), ambos `[T, N, 4]`.

---

9. Bloco 7 — Normalização das arestas (Mudança 1)

```
def normalize_edge_weights_by_destination(edge_index, edge_weight, num_nodes):
    dst = edge_index[1]
    deg = scatter(edge_weight, dst, dim=0, dim_size=num_nodes, reduce="sum")
    deg = deg.clamp(min=1e-12)
    return edge_weight / deg[dst]
```

Problema que resolve: na 0.7.5, os pesos eram normalizados por **min-max global** `(w - w.min())/(w.max() - w.min())`. Uma única aresta curtíssima (peso enorme, o caso Butantã↔IAG-USP) comprimia todos os outros pesos para perto de zero, **dominando a agregação**.

Solução ($D^{-1}A$ — normalização por linha): divide o peso de cada aresta pela **soma dos pesos que chegam ao nó de destino**. Assim, para todo nó `j`: $\Sigma_k \tilde{w}_{kj} = 1$. Cada nó recebe uma **média ponderada convexa** dos vizinhos.

$$\tilde{w}_{ij} = \frac{w_{ij}}{\sum_{k \in N(j)} w_{kj}}$$

`torch_geometric.utils.scatter` (função PyG não-nativa): é a operação de "espalhar e reduzir". Dado um vetor de valores (`edge_weight`) e um vetor de índices de destino (`dst`), agrupa os valores pelo índice e aplica a redução (`reduce="sum"`). Aqui calcula o **grau ponderado** de cada nó (soma dos pesos de entrada). `dim_size=num_nodes` garante saída com um slot por nó. `deg[dst]` faz *broadcast* do grau de volta para cada aresta. `.clamp(min=1e-12)` evita divisão por zero em nós isolados.

```
def normalize_edge_weights_symmetric(edge_index, edge_weight, num_nodes):
    deg = scatter(edge_weight, dst, ..., reduce="sum")
    deg_inv_sqrt = deg.pow(-0.5)
    return deg_inv_sqrt[src] * edge_weight * deg_inv_sqrt[dst]
```

Versão simétrica $D^{-1/2}AD^{-1/2}$ — a mesma normalização usada em GCNs. É usada **só** pelo Feature Propagation (Bloco 9), porque a teoria de minimização da energia de Dirichlet exige a adjacência simétrica. `deg.pow(-0.5)` é o elemento da matriz diagonal $D^{-1/2}$; multiplicar pela origem e pelo destino implementa a forma simétrica.

---

10. Bloco 8 — Grafo PyTorch Geometric

```
node_to_idx = {node: i for i, node in enumerate(nodes_lista)}
...
for u, v, attrs in G.edges(data=True):
```

```
    edges_src.append(i); edges_dst.append(j)
    if not G.is_directed():
        edges_src.append(j); edges_dst.append(i)   # aresta reversa
```

Conversão NetworkX → PyG. O PyG não usa um objeto "grafo" como o NetworkX; ele representa a topologia por um **`edge_index`**: um tensor `[2, E]` onde a linha 0 são os nós de origem e a linha 1 os de destino. Como o grafo é não-direcionado, cada aresta é duplicada nos dois sentidos (a mensagem precisa fluir nas duas direções).

```
g_pyg = Data(
    edge_index=edge_index,
    edge_weight=edge_weight,            # D⁻¹A (usado pelo GraphSAGE)
    edge_weight_sym=edge_weight_sym,    # D⁻¹ᐟ²AD⁻¹ᐟ² (usado pelo FP)
    edge_weight_raw=edge_weight_raw,    # 1/distância sem normalizar
    edge_distance_km=edge_distance_km,
    num_nodes=len(nodes_lista)
).to(DEVICE)
```

`torch_geometric.data.Data` é o contêiner padrão do PyG para um grafo. Aceita atributos arbitrários (`edge_weight`, `edge_weight_sym`, etc.) e sabe movê-los todos para o dispositivo com `.to(DEVICE)`. É o objeto passado ao modelo no `forward`.

---

11. Bloco 9 — Feature Propagation (Mudança 2)

Esta é a contribuição teórica central da versão. Substitui a antiga imputação por média simples dos vizinhos.

Teoria (Rossi et al., ICLR 2022)

O **Feature Propagation** preenche features faltantes minimizando a **energia de Dirichlet** do grafo:

\ell(x, G) = \tfrac{1}{2}\, x^\top \Delta x = \tfrac{1}{2} \sum_{ij} \tilde{a}_{ij}\,(x_i - x_j)^2

Intuição: a energia é baixa quando nós **vizinhos têm valores parecidos** (suavidade no grafo). Minimizá-la, mantendo fixos os valores conhecidos (âncoras), equivale a uma **difusão de calor** que "espalha" os valores observados pelos nós faltantes. O algoritmo:

1. propaga as features pela adjacência simétrica normalizada $\tilde{A} = D^{-1/2}AD^{-1/2}$;
2. **restaura os valores reais das âncoras a cada iteração** (passo crítico — sem isso, tudo convergiria para uma constante);
3. repete (aqui, **40 iterações**).

Implementação vetorizada

```
def feature_propagation_batch(X, edge_index, edge_weight, known_mask, num_iter=40):
    X = X.clone().float()
    X_known = X[:, known_mask, :].clone()        # guarda as âncoras
    src, dst = edge_index[0], edge_index[1]
    ew = edge_weight.float().view(1, -1, 1)       # [1, E, 1] p/ broadcast
    T, N, F = X.shape
```

```
    for _ in range(num_iter):
        msgs = X[:, src, :] * ew                  # [T, E, F]  difusão
        X_new = torch.zeros(T, N, F, device=X.device)
        X_new.index_add_(1, dst, msgs)            # acumula no destino
        X_new[:, known_mask, :] = X_known         # restaura âncoras
        X = X_new
    return X
```

Pontos-chave: - Processa **todos os timesteps em paralelo** (eixo `T` vetorizado), sem loop sobre o tempo — muito mais rápido. - `known_mask` é um tensor booleano `[N]`: `True` nas âncoras (nós com dado), `False` nos cegos. - `X[:, src, :]` indexa, para cada aresta, as features do nó de **origem** → tensor `[T, E, F]`. Multiplicar por `ew` aplica o peso da aresta.

`Tensor.index_add_(dim, index, source)` (PyTorch, função-chave): acumula (soma) as fatias de `source` nas posições de `index` ao longo da dimensão `dim`. Aqui, `dim=1` (nós) e `index=dst`: cada mensagem `msgs[:, e, :]` é somada ao nó de destino `dst[e]`. É a operação de **agregação por soma** dos vizinhos — equivalente a um produto matriz-esparsa ÃX feito manualmente. O sufixo `_` indica operação *in-place*.

Baseline para comparação

```
def imputacao_media_vizinhos_batch(X, edge_index, num_nodes, blind_idx):
    cont.index_add_(0, dst, torch.ones(...))      # nº de vizinhos por nó
    soma.index_add_(1, dst, X[:, src, :])          # soma das features
    media = soma / cont
    X[:, blind_t, :] = media[:, blind_t, :]        # só nos nós cegos
```

O **baseline** é exatamente uma única iteração de difusão por **média simples** (sem pesos, sem restaurar âncoras). Equivale ao método antigo. É calculado e guardado **antes** do FP para comparação quantitativa (Bloco 15) — mostrando o ganho do FP sobre a média simples.

```
X_2023 = feature_propagation_batch(X_2023, ei, ew_sym, known_mask, num_iter=40)
X_2024 = feature_propagation_batch(X_2024, ei, ew_sym, known_mask, num_iter=40)
```

O FP é aplicado tanto ao treino quanto ao teste, **antes** de entrar na GNN. Note que usa `edge_weight_sym` (a versão simétrica).

---

12. Bloco 10 — Janelamento temporal e máscara de loss

```
TARGET_IDX_PLU = 0   # índice da feature PLU
TARGET_IDX_TEMP = 1  # índice da feature Temp

Y_2023      = X_2023[1:, :, TARGET_IDX_PLU]   # alvo: PLU em t+1
X_2023_final = X_2023[:-1, :, :]               # entrada: estado em t
```

Formulação da previsão um-passo-à-frente: a entrada é o estado completo `[N, 4]` no instante `t`; o alvo é a variável-alvo no instante `t+1`. O *slicing* `[1:]` (alvo deslocado um passo) vs `[:-1]` (entrada) implementa esse pareamento. Treina em 2023, testa em 2024.

São criados **dois conjuntos**: um para PLU (`TARGET_IDX_PLU=0`) e um para Temperatura (`TARGET_IDX_TEMP=1`), pois há **dois modelos separados**.

```
IDX_TREINO = torch.tensor(
    [i for i, n in enumerate(nodes_lista) if str(n) not in NOS_SEM_DADOS],
    dtype=torch.long)
```

Máscara de loss: os nós cegos (33, 34, 35) **participam da agregação** (passam mensagens), mas seus erros **não entram no cálculo do loss** — afinal não há observação real para eles em 2023. `IDX_TREINO` lista os índices dos nós com dado; o treino só mede o erro nesses (ver Bloco 12). O modelo aprende a estimá-los **indiretamente**, via os pesos compartilhados aprendidos nos nós com dados.

---

13. Bloco 11 — A camada `WeightedSAGEConv` e o modelo GraphSAGE

Teoria (GraphSAGE, Hamilton et al. NeurIPS 2017)

GraphSAGE atualiza a representação de cada nó combinando **sua própria feature** com uma **agregação das features dos vizinhos**:

h'*i* = *W*{\text{self}}\, x_i + W_{\text{neigh}} \cdot \text{mean}*{j \in N(i)}\big(w*{ij}\, x_j\big)

A novidade aqui é a agregação ser uma **média ponderada por `edge_weight`** (os pesos geográficos normalizados), não uma média uniforme.

A camada como subclasse de `MessagePassing`

```
class WeightedSAGEConv(MessagePassing):
    def __init__(self, in_channels, out_channels):
        super().__init__(aggr="add")
        self.lin_self = nn.Linear(in_channels, out_channels)
        self.lin_neigh = nn.Linear(in_channels, out_channels)
```

`torch_geometric.nn.MessagePassing` é a classe-base do PyG para qualquer camada de GNN. Ela implementa o *esquema de passagem de mensagens* em três fases — **message** → **aggregate** → **update** — e cuida do roteamento eficiente pelas arestas. Quem herda dela só precisa definir o que cada fase faz. `aggr="add"` diz que a agregação é por **soma** (a média é feita à mão dividindo pelo grau, logo abaixo).

`nn.Linear(in, out)` é uma camada densa `y = xWᵀ + b` (as matrizes Wself e Wneigh aprendidas).

```
    def forward(self, x, edge_index, edge_weight=None):
        row, col = edge_index
        deg = torch.zeros(x.size(0), ...)
        deg.index_add_(0, col, edge_weight)        # grau ponderado (entrada)
        deg = deg.clamp(min=1e-8)

        neigh_agg = self.propagate(edge_index, x=x, edge_weight=edge_weight)
        neigh_mean = neigh_agg / deg.unsqueeze(-1)  # soma → média ponderada
```

```
        out = self.lin_self(x) + self.lin_neigh(neigh_mean)
        return out

    def message(self, x_j, edge_weight):
        return edge_weight.view(-1, 1) * x_j
```

Como o `MessagePassing` funciona aqui: - **`self.propagate(edge_index, x=x, edge_weight=...)`** — método do PyG que dispara o pipeline. Internamente, para cada aresta ele monta `x_j` (feature do nó **origem** da aresta), chama `message`, e agrega o resultado nos nós de destino com a redução `aggr="add"`. - **`message(self, x_j, edge_weight)`** — define a mensagem de cada aresta: a feature do vizinho **ponderada pelo peso** `w_ij · x_j`. O sufixo `_j` é uma convenção do PyG: `x_j` é a feature do nó de origem de cada aresta (o vizinho); `x_i` seria a do destino. O PyG monta esses tensores automaticamente a partir do `edge_index`. - A soma das mensagens (`neigh_agg`) é dividida pelo **grau ponderado** `deg` (calculado com `index_add_`), transformando soma em **média ponderada**. Isso reproduz exatamente a soma dos pesos normalizados = 1 da Mudança 1.

O modelo de 2 camadas

```
class MeteorologiaGNN(nn.Module):
    def __init__(self, in_feats, h_feats):
        self.conv1 = WeightedSAGEConv(in_feats, h_feats)   # 4 → 32
        self.conv2 = WeightedSAGEConv(h_feats, 1)          # 32 → 1
    def forward(self, data, in_feat):
        h = self.conv1(x, edge_index, edge_weight)
        h = F.relu(h)
        h = self.conv2(h, edge_index, edge_weight)
        return h.squeeze(-1)
```

Arquitetura **4 → 32 → 1**: duas camadas `WeightedSAGEConv` com **ReLU** entre elas (`F.relu` zera valores negativos, introduzindo não-linearidade). Duas camadas = cada nó "enxerga" até **2 saltos** de distância no grafo (seus vizinhos e os vizinhos dos vizinhos). A saída `[N, 1]` é achatada com `squeeze(-1)` para `[N]` — uma previsão escalar por nó. `nn.Module` é a classe-base de todo modelo PyTorch; `__init__` registra as camadas (e seus parâmetros treináveis) e `forward` define o fluxo de dados.

---

14. Bloco 12 — Treino

```
def treinar_modelo(X_train, Y_train, g_pyg, idx_treino, ...):
    modelo = MeteorologiaGNN(in_feats, h_feats).to(DEVICE)
    optimizer = torch.optim.Adam(modelo.parameters(), lr=lr)
    criterion = nn.MSELoss()
    ...
    for epoch in range(1, max_epochs + 1):
        for i in range(len(X_train)):
            pred = modelo(g_pyg, X_train[i])
            loss = criterion(pred[idx], Y_train[i][idx])   # só nós com dados
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
```

Funções de biblioteca: - `torch.optim.Adam(params, lr)` — otimizador Adam (adaptive moment estimation): ajusta a taxa de aprendizado por parâmetro usando médias móveis do gradiente e do seu quadrado. Padrão moderno, converge rápido. `modelo.parameters()` entrega todos os pesos treináveis. - `nn.MSELoss()` — erro quadrático médio, a função de perda de regressão. - `optimizer.zero_grad()` — zera os gradientes acumulados (PyTorch acumula por padrão; sem isso, gradientes de iterações anteriores somariam). - `loss.backward()` — *backpropagation*: calcula o gradiente da loss em relação a cada parâmetro via autograd. - `optimizer.step()` — aplica um passo de descida do gradiente, atualizando os pesos.

Máscara aplicada: `criterion(pred[idx], y_i[idx])` calcula a loss **apenas nos nós com dado** (`idx = IDX_TREINO`). Os nós cegos influenciam `pred` (pela agregação), mas não penalizam o gradiente.

Parada antecipada (early stopping): o loop guarda o melhor estado (`deepcopy(modelo.state_dict())`) e para se (a) o erro cai abaixo de `target_loss`, ou (b) não melhora mais que `min_delta` por `patience` épocas consecutivas. Ao final, restaura o melhor estado com `load_state_dict`.

- `modelo.state_dict()` — dicionário com todos os tensores de pesos do modelo. `deepcopy` garante uma cópia independente (senão guardaria uma referência que muda junto). `load_state_dict` recarrega esses pesos.
- `modelo.train()` / `modelo.eval()` alternam o modo (afeta camadas como dropout/batchnorm — aqui não há, mas é boa prática).

Treina-se **dois modelos**: `modelo` (pluviosidade) e `modelo_temp` (temperatura), ambos com `max_epochs=50`, `lr=0.001`.

---

15. Bloco 13 — Avaliação e gráficos

`avaliar_e_plotar_todos_nos` roda o modelo em todo o teste e gera um gráfico por nó.

```
modelo.eval()
with torch.no_grad():
    for i in range(len(X_test)):
        pred = modelo(g_pyg, X_test[i])
        preds_all.append(pred.cpu().numpy().reshape(-1))
        trues_all.append(Y_test[i].cpu().numpy().reshape(-1))
```

`with torch.no_grad():` desliga o autograd durante a inferência — não precisa de gradientes, então economiza memória e acelera. `.detach().cpu().numpy()` solta o tensor do grafo de computação, move para CPU e converte para NumPy (matplotlib não lê tensores na GPU).

Métricas por nó: - `mae = mean(|pred − true|)` — erro absoluto médio. - `rmse = sqrt(mean((pred − true)²))` — raiz do erro quadrático médio.

Janela de plotagem: centra o gráfico no evento mais relevante de cada nó: - `criterio_janela="pico_real"` → `np.argmax(trues_no)` (o pico de chuva) — usado para PLU. -

`criterio_janela="maior_desvio_media"` → `np.argmax(|trues − média|)` (maior desvio) — usado para Temp.

Os gráficos são salvos em `graficos_2024/pluviosidade/` e `graficos_2024/temperatura/`. Funções auxiliares (`plotar_no_especifico`, `plotar_no_mes`, `plotar_nos_cegos_janeiro`, `plotar_media_todos_nos_janeiro_2024`) fazem recortes específicos — com destaque para os **nós cegos em janeiro/2024**, onde se compara a estimativa do modelo com a referência.

---

16. Bloco 14 — Identificação de anomalias (Mudança 3)

Duas regras complementares.

Regra 1 — Limiar meteorológico previsto (alerta antecipado)

```
def compute_thresholds(X_historical, timestamps, percentile=95):
    months = np.array([t.month for t in pd.to_datetime(list(timestamps))])
    tau = np.zeros((12, N, Fd))
    for m in range(12):
        mask = months == (m + 1)
        tau[m] = np.percentile(X_np[mask], percentile, axis=0)
    return tau
```

Pré-computa `τ[mês, nó, variável]` = **percentil 95** histórico (sobre 2023). **`np.percentile(..., 95, axis=0)`** devolve o valor abaixo do qual ficam 95% das observações — o limiar do que é "anormalmente alto" para aquela estação naquele mês. O cálculo por mês captura sazonalidade.

```
def rule1_alert(X_hat, tau, month):
    return X_hat > tau[month - 1]
```

Dispara alerta quando a **previsão** $\hat{x}$ ultrapassa o limiar. Como age sobre a previsão (e não sobre a observação), é um **alerta antecipado**.

```
def aplicar_janela_antiruido(alert_series, w_alert=3):
    for t in range(T):
        run = np.where(alert_series[t], run + 1, 0)   # contador de consecutivos
        confirmado[t] = run >= w_alert
    return confirmado
```

Janela anti-ruído: só confirma o alerta após **3 ativações consecutivas**. `np.where(cond, a, b)` é um if vetorizado: incrementa o contador `run` onde houve alerta, zera onde não houve. Evita falsos positivos por *spikes* isolados de sensor.

Regra 2 — Anomaly Score multivariado (Mahalanobis)

```
def compute_sigma_inv(X_historical):
    for i in range(N):
        xi = X_np[:, i, :]
        cov = np.cov(xi.T)
```

```
        sigma_inv[i] = np.linalg.pinv(cov)
    return sigma_inv
```

Para cada estação, estima a **matriz de covariância** histórica das variáveis e a inverte. - **`np.cov(xi.T)`** — matriz de covariância `[F, F]` entre as variáveis (espera variáveis nas linhas, por isso `.T`). - **`np.linalg.pinv(cov)`** — **pseudo-inversa de Moore-Penrose**. Usa-se a pseudo-inversa (e não `inv`) porque a covariância pode ser singular/mal-condicionada (variáveis correlacionadas), o que quebraria a inversão comum.

```
def compute_anomaly_score(x_real, x_hat, sigma_inv):
    e = x_real - x_hat
    return torch.einsum("nf,nfg,ng->n", e, sigma_inv, e).clamp(min=0).sqrt()
```

Teoria — distância de Mahalanobis: mede quão "estranho" é o erro `e = x_real − x̂` levando em conta a **correlação** entre variáveis e a escala de cada uma:

AS_{i} = \sqrt{e^\top \Sigma_i^{-1} e}

Um erro grande numa variável muito estável (baixa variância) pesa mais que o mesmo erro numa variável volátil. É **multivariada**: combina PLU, Temp, Umid, Vel num único score por estação.

`torch.einsum("nf,nfg,ng->n", e, sigma_inv, e)` — *Einstein summation*, uma notação compacta para produtos tensoriais. A string descreve os índices: - `e` tem índices `nf` (nó, feature), - `sigma_inv` tem `nfg` (nó, feature, feature), - o segundo `e` tem `ng`, - a saída é `n` (só o nó).

Isso calcula, **para cada nó `n` em paralelo**, a forma quadrática `eᵀ Σ⁻¹ e = Σ_f Σ_g e_f · σ⁻¹_{fg} · e_g`. `.clamp(min=0)` protege contra pequenos negativos numéricos antes do `.sqrt()`.

Orquestração

O código calcula `tau` e `sigma_inv` sobre 2023, gera previsões multivariadas `X_hat_mv` em 2024 (PLU e Temp pelos modelos; demais variáveis por persistência da entrada), aplica as duas regras e consolida uma **tabela de alertas** `df_alertas` com estação, variável, score e timestamp. `np.where(alert_confirmado)` retorna os índices `(t, n, f)` onde há alerta confirmado.

---

17. Bloco 15 — Métricas e baseline

```
for j in blind_idx:
    for fdim, fname in enumerate(FEATURE_NAMES):
        mae_b, rmse_b = mae_rmse(X2024_np[:, j, fdim], X2024_base_np[:, j, fdim])
```

Compara, **nos nós cegos**, a estimativa por **Feature Propagation** (`X_2024`) com o **baseline de média simples** (`X_2024_baseline`), variável a variável — quantificando o ganho da Mudança 2. Também resume o MAE/RMSE médio de previsão (FP + GraphSAGE) por variável no teste.

---

18. Bloco 16 — Exportação de artefatos

```
torch.save(modelo.state_dict(), ".../model_plu.pt")
torch.save(modelo_temp.state_dict(), ".../model_temp.pt")
pickle.dump(tau, ...); pickle.dump(sigma_inv, ...)
pickle.dump(graph_data, ...); pickle.dump(X_test_export, ...)
```

Salva tudo o que o dashboard precisa:

Arquivo	Conteúdo
`model_plu.pt`, `model_temp.pt`	pesos dos dois modelos (`state_dict`)
`tau.pkl`	limiares P95 `[12, N, F]`
`sigma_inv.pkl`	inversas de covariância `[N, F, F]`
`graph_data.pkl`	`edge_index`, pesos, coords, labels, nós cegos, nomes de features
`X_test.pkl`	tensores de teste, previsões, alertas, scores e DataFrames de métricas

- **`torch.save(obj, path)`** — serializa tensores/dicionários PyTorch (usa pickle internamente). O `state_dict` é o que se recarrega depois com `load_state_dict`.
- **`pickle.dump(obj, f)`** — serialização genérica do Python para objetos arbitrários (arrays NumPy, DataFrames, dicionários).

---

19. `dashboard.py` — visualização

Aplicativo **Streamlit** que **não treina nada**: ele apenas carrega os artefatos e visualiza. (Importante: o dashboard **não reconstrói a GNN** — usa as previsões `X_hat_mv` já calculadas e salvas pelo notebook. Não há inferência ao vivo.)

```
@st.cache_data(show_spinner=True)
def carregar_artefatos():
    ...
```

`@st.cache_data` — decorador do Streamlit que **memoriza** o resultado da função: os `.pkl` são lidos do disco uma vez só, não a cada interação do usuário. Essencial para performance, já que o Streamlit re-executa o script inteiro a cada clique.

Cinco abas (`st.tabs`):

1. **Grafo** — mapa geográfico das estações com `plotly.graph_objects.Scattermapbox`. As arestas viram linhas (com `None` separando segmentos), os nós viram marcadores coloridos por estado (verde=normal, vermelho=alerta, cinza=cego). `mapbox_style="carto-positron"` é o mapa-base.
2. **Monitoramento** — série real (t+1) vs prevista para a estação/variável escolhida, com **banda P5–P95** (`add_hrect`) do mês e marcadores `×` nos alertas da Regra 1.

3. **Anomaly Score** — tabela `df_alertas` ordenada por score + **heatmap** estação×variável da contribuição de cada variável ao score de Mahalanobis (`|e_k|·√σ⁻¹_kk`), via `plotly.express.imshow`.
4. **Nós Cegos** — compara, para cada nó cego, a estimativa **FP + GNN** vs o **baseline** de média dos vizinhos, com MAE/RMSE.
5. **Métricas do Modelo** — tabelas de MAE/RMSE por nó (chuva e temperatura) e o resumo por variável.

Funções Streamlit-chave: `st.sidebar.slider` (escolhe o timestep "atual"), `st.selectbox`, `st.dataframe`, `st.metric`, `st.plotly_chart`, `st.cache_data`.

---

20. Glossário de funções de biblioteca não-nativas

NetworkX

Função	O que faz
`nx.read_graphml`	Lê grafo de arquivo GraphML (XML), com atributos de nós/arestas
`G.edges(data=True / keys=True)`	Itera arestas com atributos / com a chave do multigrafo
`G.neighbors(n)`	Vizinhos de um nó
`nx.draw`	Desenha o grafo com matplotlib

PyTorch

Função	O que faz
`torch.device`, `.to(device)`	Define/move tensores para CPU/CUDA/MPS
`torch.tensor`, `torch.zeros`, `.clone()`	Cria/copia tensores
`Tensor.index_add_(dim, idx, src)`	Acumula `src` nas posições `idx` (agregação por soma, in-place)
`torch.einsum("...", a, b, c)`	Produto tensorial por notação de Einstein (formas quadráticas, etc.)
`.clamp(min=)`, `.pow()`, `.sqrt()`	Operações elementares com proteção numérica
`.squeeze(-1)`, `.view()`, `.transpose()`	Reorganizam dimensões do tensor
`nn.Module`, `nn.Linear`, `F.relu`	Base de modelos, camada densa, ativação
`nn.MSELoss`, `torch.optim.Adam`	Função de perda e otimizador
`loss.backward()`, `optimizer.step()`, `zero_grad()`	Ciclo de treino (autograd + atualização)
`state_dict()`, `load_state_dict()`, `torch.save`	Salvar/restaurar pesos
`with torch.no_grad()`	Desliga autograd na inferência

PyTorch Geometric (PyG)

Função	O que faz
`Data(...)`	Contêiner de um grafo (edge_index + atributos)
`MessagePassing`	Classe-base de camadas GNN (message→aggregate→update)
`self.propagate(...)`	Dispara o pipeline de passagem de mensagens
`message(x_j, ...)`	Define a mensagem por aresta (`x_j` = feature do vizinho)
`aggr="add"`	Define a agregação (soma)
`utils.scatter(src, index, reduce=)`	Agrupa-e-reduz por índice (grau ponderado, etc.)
`edge_index [2, E]`	Topologia: linha 0 = origem, linha 1 = destino

pandas

Função	O que faz
`pd.read_csv` (engine="python", on_bad_lines)	Leitura tolerante de CSV sujo
`pd.to_datetime` (format="mixed", dayfirst)	Parse robusto de datas mistas
`pivot_table`	Long → wide com MultiIndex `(feature, nó)`
`MultiIndex.from_product` , `reindex`	Garante todas as combinações feature×nó
`interpolate(method="time")`	Preenche faltantes pelo tempo
`ffill()` , `bfill()`	Preenche para frente/trás
`.str.replace` , `pd.to_numeric`	Limpeza de vírgula decimal e conversão numérica
`index.duplicated` , `set_index` , `sort_index` , `concat`	Manipulação de índice temporal

NumPy

Função	O que faz
`np.percentile(x, 95, axis=0)`	Limiar P95 (Regra 1)
`np.cov` , `np.linalg.pinv`	Covariância e pseudo-inversa (Mahalanobis)
`np.where(cond, a, b)`	If vetorizado (janela anti-ruído)
`np.argmax` , `reshape` , `transpose`	Índice do máximo e remodelagem de arrays

Outras

Função	O que faz
`unicodedata.normalize("NFKD")`	Remove acentos (normalização de nomes)

Função	O que faz
`re.sub`, `re.match`	Regex para limpeza/casamento de tokens
`io.StringIO`	"Arquivo em memória" a partir de string
`pickle.dump/load`	Serialização de objetos Python
`streamlit` (`st.tabs`, `st.cache_data`, ...)	UI do dashboard
`plotly` (`Scattermapbox`, `imshow`, `add_hrect`)	Mapa, heatmap e bandas

Referências teóricas

- **Hamilton, Ying & Leskovec.** *Inductive Representation Learning on Large Graphs* (GraphSAGE). NeurIPS 2017.
- **Rossi et al.** *On the Unreasonable Effectiveness of Feature Propagation in Learning on Graphs With Missing Node Features.* ICLR 2022.
- **Distância de Mahalanobis** — P. C. Mahalanobis, 1936.
- **Normalização $D^{-1}A$ / $D^{-1/2}AD^{-1/2}$** — formulação clássica de GCN (Kipf & Welling, 2017).

Documento gerado a partir da análise de `grafo-0.8.0-pyg.ipynb` e `dashboard.py`.